ENTREGA OBLIGATORIA N.º 1 · 1.º PARCIAL

LogiRed

Plataforma Distribuida de Logística de Última Milla

Grupo 13: Facundo Tisch · Lola Díaz · Nicolás Valenzuela
Materia: Desarrollo de Aplicaciones II · Comisión Lunes TM
Universidad Argentina de la Empresa (UADE) · Cursada 2026

DISEÑO ARQUITECTÓNICO

Arquitectura Global en Tres Capas

CAPA 1

Presentación (Frontend)

Tecnología: React 19 + Vite (SPA).

Responsabilidad: Captura de eventos, renderizado reactivo del Dashboard de despacho, selector dinámico de rutas y consola de stock. No ejecuta reglas de negocio (Skinny UI).

CAPA 2

Negocio (Backend)

Tecnología: Spring Boot 3.5.0 (Java 17).

Responsabilidad: Contenedor IoC/DI. Orquesta el ciclo de vida de los beans, transacciones declarativas (@Transactional), seguridad RBAC y algoritmos de ruteo dinámico.

CAPA 3

Persistencia (Datos)

Tecnología: PostgreSQL 18 + Spring Data JPA.

Responsabilidad: Persistencia relacional con aislamiento ACID. Manejo de claves foráneas, bloqueos de concurrencia e integridad transaccional definitiva de pedidos e inventario.

DETALLE DE CAPA

1. Capa de Presentación (Frontend React)

Ecosistema Tecnológico

REACT 19 Interfaz declarativa basada en componentes modulares independientes.

VITE 8.2 Herramienta de compilación ultrarrápida con Hot Module Replacement (HMR).

REACT ROUTER 7 Enrutamiento dinámico en el cliente y protección de rutas privadas.

AXIOS / FETCH Clientes HTTP asincrónicos para consumo de servicios REST.

Rol Arquitectónico (Skinny UI)

• **Single Page Application (SPA):** Flujo interactivo sin recarga total de página, optimizando la experiencia del operador logístico.

• **Desacoplamiento Estricto:** La interfaz jamás toma decisiones de negocio ni accede a persistencia directa. Toda lógica (validación de stock, tarifas, ruteo) se delega a la Capa de Negocio.

• **Gestión de Sesión:** Contexto global (AuthContext) que conserva las credenciales seguras para dialogar con el backend.

DECISIONES TECNOLÓGICAS

Justificación del Stack Tecnológico

FRONTEND

React 19 + Vite

Elegido por su enfoque declarativo por componentes. Permite sincronizar vistas complejas de despacho y dashboards multidepósito sin recargar la pantalla, facilitando el mantenimiento y la extensibilidad del frontend sin acoplarlo a una tecnología específica de backend.

BACKEND

Spring Boot 3.5.0

Elegido por su contenedor de Inversión de Control (IoC). Automatiza la inyección de dependencias, resuelve aspectos no funcionales mediante proxies (seguridad declarativa y transacciones @Transactional) y provee un servidor embebido de alto rendimiento para agilizar el despliegue.

PERSISTENCIA

PostgreSQL 18

Elegido como motor relacional transaccional estricto (descartando bases en memoria volátiles como H2). Garantiza el cumplimiento de propiedades ACID, aislamiento de concurrencia y consistencia física obligatoria para la logística de depósitos.

CICLO DE VIDA EN EL CONTENEDOR

Gestión de Estado: Stateful vs. Stateless

STATELESS (SIN ESTADO)

PedidoService y RuteoService

• **Ámbito:** Singleton (@Service por defecto).

• **Comportamiento:** Idempotente y thread-safe. Ningún dato del cliente se conserva en memoria del servidor entre peticiones.

• **Escalabilidad:** Arquitectura "Share Nothing". Permite escalado horizontal elástico sin necesidad de afinidad de sesiones.

• **Analogía:** Como una calculadora o cajero automático. Realiza la operación y queda disponible para el siguiente usuario sin memoria previa.

STATEFUL (CON ESTADO)

InventarioService (@SessionScope)

• **Ámbito:** Una instancia por sesión HTTP activa.

• **Comportamiento:** Mantiene un contexto conversacional vivo en memoria del servidor asociado exclusivamente al operador logístico.

• **Escalabilidad:** Requiere Sticky Sessions en el balanceador o externalización del estado a una caché distribuida (Redis).

• **Analogía:** Como un changuito de compras físico. Retiene temporalmente los ítems mientras dura el proceso de compra.

COMPONENTE STATEFUL

El "Hold" de Stock: InventarioService

Justificación Operativa (5 Minutos)

En logística urbana con depósitos físicos de consolidación (Barracas y Munro), permitir que múltiples operadores carguen órdenes simultáneamente sin bloquear stock produce **sobreventa accidental**.

`InventarioService` implementa un hold preventivo: al reservar stock, se descuenta de lo disponible y pasa a stock bloqueado en PostgreSQL, asignando la reserva a la sesión del usuario.

Gestión de Ciclo de Vida por el Contenedor

@POSTCONSTRUCT Al iniciar la sesión conversacional, genera un identificador único de sesión y precarga el catálogo si la base está vacía.

@PREDESTROY (LIBERACIÓN AUTOMÁTICA) Si el operador cierra el navegador o expira la sesión HTTP, el contenedor ejecuta este callback automáticamente. El servidor reintegra todas las unidades bloqueadas al stock disponible en PostgreSQL sin intervención humana.

COMPONENTES DE NEGOCIO

Servicios Stateless del Dominio

COMPONENTE 1

PedidoService

• **Responsabilidad:** Administra el ciclo de vida completo de las órdenes de envío (alta, cotización inicial, actualización de estado y baja definitiva).

• **Transaccionalidad:** Opera bajo demarcación declarativa `@Transactional`. Si ocurre un fallo en cascada, el contenedor revierte todas las modificaciones físicas.

• **Aislamiento:** No guarda datos conversacionales; cada petición recibe su identificador de pedido y procesa de forma autocontenida.

COMPONENTE 5

RuteoService

• **Responsabilidad:** Cálculo matemático y ordenamiento de secuencias de entrega eficientes a partir de matrices de paradas.

• **Servicio SOA Reutilizable:** Es un servicio puro que no contamina la base de datos. Es consumido por el comercio al cotizar, por el repartidor al recibir su hoja de ruta y por el módulo de transportistas externos.

• **Idempotencia Total:** Mismas coordenadas de entrada producen siempre la misma ruta calculada.

FLUJO INTEGRADO

Anatomía de una Petición: "Crear Pedido"

1 **Presentación (React SPA)**

El usuario confirma el formulario de envío; React despacha un HTTP POST `/api/pedido` con el payload JSON.

2 **Controlador REST (PedidoController)**

Deserializa el JSON en un objeto DTO, valida formato y delega a la capa de servicio (Skinny Controller).

3 **Capa de Negocio (PedidoService - Facade)**

El contenedor abre el contexto transaccional (`@Transactional`). Se validan tarifas y estado inicial "CREADO".

4 **Capa de Acceso a Datos (PedidoRepository - DAO)**

Spring Data JPA traduce la entidad Pedido a una instrucción SQL `INSERT INTO pedidos` mediante Hibernate.

5 **Persistencia Transaccional (PostgreSQL 18)**

La base confirma físicamente el commit ACID, asigna el ID autonumérico y el backend retorna **HTTP 200 OK**.

PATRONES DE DISEÑO (1 DE 3)

Patrón DAO / Repository

Problema y Solución

• **Problema:** Escribir consultas SQL directamente en la lógica de negocio acopla el código al motor de base de datos y genera código repetitivo (conexiones JDBC, mapeo manual de filas a clases).

• **Solución:** El patrón DAO (Data Access Object) encapsula todos los detalles técnicos del almacenamiento detrás de una interfaz fuertemente tipada.

Implementación en LogiRed

• `PedidoRepository` e `InventarioRepository` extienden de `JpaRepository<T, ID>`.

• **Beneficio de Ingeniería:** Cumple el Principio de Responsabilidad Única (SRP). La capa de servicio manipula únicamente entidades de dominio (`Pedido`, `ItemInventario`) abstrayéndose por completo de las sentencias SQL y garantizando portabilidad.

PATRONES DE DISEÑO (2 DE 3)

Patrón Strategy: Ruteo Dinámico

Problema y Solución

• **Problema:** En logística urbana, el algoritmo óptimo depende del tipo de vehículo y la hora: un furgón pesado requiere minimizar kilómetros, mientras que una moto prioriza evadir congestión.

• **Solución:** Encapsular los algoritmos en una familia de clases intercambiables que implementan una interfaz común.

Implementación en LogiRed

• **Interfaz:** `EstrategiaRuteo`.

• **Implementaciones Concretas:**
- `RutaMasCortaStrategy`: Minimiza kilometraje acumulado.
- `RutaMasRapidaStrategy`: Pondera coeficientes de velocidad y tráfico urbano.

• **Principio OCP:** Se pueden incorporar nuevas estrategias (ej. *RutaEcoFriendly*) sin modificar `RuteoService`.

PATRONES DE DISEÑO (3 DE 3)

Patrón Facade: Service Layer

Problema y Solución

• **Problema:** Un controlador web expuesto al caos interno del sistema (manejo de transacciones, orquestación de algoritmos, llamadas a múltiples repositorios) se transforma rápidamente en un "Fat Controller" inmanejable.

• **Solución:** Disponer de una fachada unificada que concentre las interacciones complejas del subsistema.

Implementación en LogiRed

• Las clases `PedidoServiceImpl`, `RuteoServiceImpl` e `InventarioServiceImpl` actúan como fachadas.

• **Skinny Controllers:** Los controladores REST solo invocan un método simple de la fachada (ej. `pedidoService.crear(dto)`). La fachada orquesta validaciones de tarifas, transacciones y persistencia internamente.

SEGURIDAD EN APLICACIONES

Seguridad Declarativa por Roles (RBAC)

ROL OPERADOR (ROLE_OPERADOR)

Permisos Operativos Habituales

• Crear nuevos pedidos y consultar el estado de despacho.
• Reservar unidades temporales en el inventario.

Intento de Operación Sensible: Si intenta ejecutar la baja física de un pedido (`DELETE /api/pedido/{id}`), Spring Security intercepta la petición por AOP antes de ejecutar el método y responde inmediatamente con un código **HTTP 403 Forbidden**.

ROL ADMINISTRADOR (ROLE_ADMIN)

Permisos Elevados (Acción Sensible)

• Protegido declarativamente con: `@PreAuthorize("hasRole('ADMIN')")`.

• El Administrador supera el filtro de autorización, ejecuta la baja definitiva del registro en PostgreSQL y el servidor devuelve **HTTP 204 No Content**.

• **Ventaja Declarativa:** Desacopla las políticas de acceso de la lógica de dominio sin ensuciar los métodos con sentencias `if` manuales.

GESTIÓN Y METODOLOGÍA

Gestión Ágil del Proyecto: Asana & Git

Distribución y Desarrollo Incremental

• **Control de Versiones (Git):** Historial de commits estructurado que evidencia el trabajo colaborativo e incremental continuo de los tres integrantes, evitando entregas monolíticas de último momento.

• **División Equitativa de Responsabilidades:**
- Facundo Tisch: Setup backend, PedidoService y Estrategias de Ruteo.
- Lola Díaz: Frontend React SPA, Dashboard y vistas interactivas.
- Nicolás Valenzuela: Seguridad declarativa RBAC, InventarioService stateful e informes técnicos.

Seguimiento en Tiempo Real

El avance del proyecto y la trazabilidad de tareas se gestionaron a través del tablero de Asana del Grupo 13.

`https://app.asana.com/1/1218020465201127/project/1218022314161278/board/1218022314235618`

Tablero con acceso público de lectura configurado para la cátedra.

EVOLUCIÓN ARQUITECTÓNICA

Roadmap de Integración

MODELO ARQUITECTÓNICO

Hub-and-Spoke

El backend Spring Boot opera como Hub concentrador central. Reduce las conexiones necesarias de $N(N-1)/2$ (15 enlaces directos) a exactamente N (6 conexiones), evitando el espagueti de integración punto a punto.

ESTRATEGIA DE ENLACE

Sincronía vs. Asincronía

• **Sincrónico (REST/SOAP):** Validación de cobertura postal y cotización inmediata.

• **Asincrónico (Colas JMS):** Desacopla la asignación de choferes (Queue punto a punto) y la notificación de estados (Topic pub/sub).

PRINCIPIOS SOA

Servicio Reutilizable

`RuteoService` se consolida bajo principios SOA: contrato explícito fuertemente tipado, reutilizado por 3 procesos clave (creación de pedidos, hoja de ruta del chofer y derivación interurbana externa).